# scripts/pre_build/check_cm_closure_seam_binds.sh â€” CM-CLOSURE-SEAM-BINDS gate

**Revision:** 1 **Last modified:** 2026-08-21T16:00:00Z **Status:** active **Item:** BOB-136 (closure seam does not bind)

## Purpose

Make the **closure seam** bind mechanically. The gate reconciles what the repositoryâ€™s own git history says happened against what the workable-items tracker says the state is, and reports â€” or, at the commit seam, **refuses** â€” when the two contradict each other.

## The defect it closes (BOB-136)

Nothing in this repo ever linked a commit back to its tracker row:

| existing check | compares |
|---|---|
| `workable-items validate` | the DB against **its own** invariants |
| `workable-items diff` | the DB against **the Markdown** |
| `docs-sync-commit-seam.sh` CHECKS 1â€“3 | the DB against **the Markdown** |
| `docs_chain verify` | the Markdown against **its exports** |

Every one of those compares the tracker to *itself*. All four are green while a row says `Queued` and git says the work landed weeks ago. Closure therefore depended on a human remembering, and demonstrably did not happen â€” a 2026-08-20 sweep found `BOB-076`, whose commit message literally says `closes BOB-076`, still `Queued`, and a 2026-08-21 sweep found 20 more.

A status field that does not move when work lands stops being evidence and becomes decoration, and every downstream reader of status silently degrades: release gates, the â€œis this owed?â€ question, and the Â§11.4.55 reopen counts used to rank the most fragile work. Â§11.4.227: an itemâ€™s done state is its **seam** landing, not its **text** landing. This gate is that seam.

## What it asserts â€” and deliberately does not

The gate never decides â€œthis item is finishedâ€. It asserts the far narrower, mechanically decidable proposition that a specific *status literal is false*:

| class | condition |
| --- | --- |
| CONTRADICTION | a message says it CLOSES/FIXES/RESOLVES <id>, yet <id> is non-terminal |
| UNRECONCILED | a **work-type** commit declares <id>, yet <id> is exactly Queued â€” which asserts *no work has started* |
| UNTRACKED-ID | a commit declares <id> and <id> has no row at all |

Remediation for UNRECONCILED is In progress **or** a close; the gate does not presume which. Statuses that already tell the truth about landed work â€” In progress, Ready for testing, In testing, Reopened, Operator-blocked, and every terminal ... (â†’ Fixed.md) â€” are never flagged by UNRECONCILED. That scoping is what keeps it from firing on a healthy tree (Â§11.4.201(1)).

# Carrier controls (Â§11.4.201(7)(a) â€” match structure, not substring)

A gate that flags correctly-tracked work is exactly as broken as one that misses stale rows: it gets switched off within a week. Links are therefore extracted **structurally** â€” conventional-commit scope token, bare-id subject prefix, closure-keyword adjacency â€” never by â€œthe message contains the stringâ€. Four carrier classes are excluded by construction, each with a fixture in the meta-test:

| id | carrier | example |
| --- | --- | --- |
| c1 | filing verb | docs(tracker): file BOB-149 â€” filing an item is not doing it |
| c2 | path component | docs/qa/BOB-117/closure-evidence.md |
| c3 | quoted text | a work commit quoting "closes BOB-076", or an indented / >-quoted pasted ticket |
| c4 | non-work type | docs(BOB-141): ..., chore, ci |

Live proof of c1: BOB-146 and BOB-149 were both *filed* on 2026-08-21 and the sweep reports neither.

Both real in-repo spellings are handled: compact runs (closes BOB-081/083/089/117, docs(BOB-124/125/126)) and suffixed scope tokens (BOB-111-followup, BOB-122-fallout). A needle exercising only the plain spelling would certify a blind query.

# Usage

```
scripts/pre_build/check_cm_closure_seam_binds.sh                  # sweep
scripts/pre_build/check_cm_closure_seam_binds.sh --report-only    # audit,
scripts/pre_build/check_cm_closure_seam_binds.sh --message-file F # COMMIT
scripts/pre_build/check_cm_closure_seam_binds.sh --message "TEXT" # COMMIT
scripts/pre_build/check_cm_closure_seam_binds.sh --self-test      # Â§11.4
```

Options: `--repo DIR` `--db PATH` `--prefix P` `--rev-range R` `-v`

# Internal behaviour

- **CHECK A â€” closure seam.** Extracts structural commitâ†”ï¸Žitem links, joins them against the tracker (read-only; the gate never writes to the DB) and classifies as above.
- **CHECK B â€” flagless-diff callers.** BOB-136 acceptance (b). Sweep mode only. `workable-items diff --db X` *without* `--issues`/`--fixed` opens zero Markdown files and still prints `DB and Markdown are in sync` â€” a Â§11.4.201(6) **false-null**. Until the shared engine refuses that invocation, no caller here may use it. Scanned over tracked executable files only: docs and this gateâ€™s own header legitimately quote the flagless form, and matching those would be the exact carrier mistake.
- **Control needle (Â§11.4.201(7)(b)).** A zero-finding verdict is worthless if the extractor is blind â€” a blind query and a clean tree return the identical quiet zero. Before reporting, the gate runs a synthetic message carrying the *same load-bearing features* as the real query (scope form, compact run, closure keyword) and aborts with exit 2 if fewer than 4/4 known links are seen.

# Commit-seam mode

Evaluates **only** the ids the pending message declares. It is monotone by construction: it can refuse the commit about to create fresh drift, and is structurally incapable of blocking on the pre-existing backlog. That is why it needs no baseline/ratchet file â€” a file which would itself rot (Â§11.4.215).

Wired via `scripts/hooks/docs-sync-commit-seam.sh` **CHECK 5**, which `scripts/commit-push-all.sh` invokes with `--message "$MSG"`. CHECK 5 is placed deliberately *before* the hookâ€™s â€œno docs-chain source stagedâ€ early exit: CHECKS 1â€“4 all trigger on a staged tracker file, but a commit landing a pure **code** fix stages no tracker file at all â€” and that is precisely the commit whose row fails to move. Running it after the early exit would make it structurally unable to see the defect it exists for (Â§11.4.196(F)). Backward-compatible: without `--message`, CHECK 5 does not run.

# Exit codes

| code | meaning |
|---|---|
| 0 | PASS, or `--report-only`, or an honest Â§11.4.3 SKIP |
| 1 | FAIL â€” findings, each named on stderr with its remediation |
| 2 | ERROR â€” usage, missing repo, or a failed control needle |

An absent DB or `sqlite3` is a loud SKIP-with-reason exiting 0: an unreadable tracker is not evidence of drift, and refusing on it would be the Â§11.4.201(1) false positive this gate exists to avoid. CHECK B still runs in that case â€” sequencing it behind the tracker check was a false-null caught by this gateâ€™s own meta-test (fixture `bad-6`).

# Honest boundary (Â§11.4.6)

The gate can only see work that **declares its id**. Work landing under a message carrying no id at all is invisible to it â€” that is BOB-136â€™s second defect verbatim, and `BOB-146` is a live instance: its fix landed under `fix(fail-open): triage all 36 Â§11.4.252 hits...`, which names no item. The gate does not claim to prevent all drift.

It also proves only that a status literal is false â€” never that an item is finished. Closure evidence remains Â§11.4.115(F) / Â§11.4.146(D3) territory, and terminal-side integrity (an evidence path that does not resolve) is already covered by `workable-items validate` (HXC-217).

# Related

- `tests/pre_build/test_check_cm_closure_seam_binds.sh` â€” Â§1.1 paired-mutation meta-test, 29 hermetic checks (golden-bad, golden-good, 5 carrier controls, commit-seam polarity).
- `scripts/hooks/docs-sync-commit-seam.sh` â€” CHECK 5 wiring.
- `scripts/commit-push-all.sh` â€” passes `--message "$MSG"` at stage 5a.